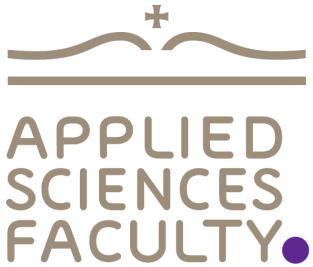
C++ — лекція 17. Шаблони — 1





©Олег Фаренюк,
Creative Commons - Attribution Share Alike (CC BY-SA)

Шаблони (**template**) — засіб C++, який призначений для кодування узагальнених алгоритмів, без прив'язки до деяких їх параметрів, таких як типи даних, розміри, тощо.

Програмування із використанням шаблонів називається узагальненим програмуванням. Воно, разом із перевантаженням функцій та операторів є різновидом статичного поліморфізму.

В C++ можна створювати шаблони класів та функцій.

Якщо для шаблону конкретизовано його параметри, генерується конкретний тип чи функція. Такий процес називається інстанціюванням (instantiation).

Шаблон функції — приклад

```
template <typename SomeType>
SomeType GetMax (SomeType a, SomeType b)
{
    return (a>b ? a:b);
}
int main(){
    double a1=5.5,b1=7.5;
    int  a2=3,b2=55;

    cout << GetMax<double>(a1,b1) << endl;
    cout << GetMax<int>(a2,b2) << endl;

    cout << GetMax(a1,b1) << endl;
    cout << GetMax(a2,b2) << endl;
      .....
}
```

Зауважте, компілятор може визначати, який тип було передано.

class та typename

В списку параметрів шаблону можна використовувати як слово **typename**, так і слово **class**. При чому, навіть якщо вжито class, параметром може бути будь який тип, не обов'язково клас.
Тобто, запис:

```
template <typename SomeType>
SomeType GetMax (SomeType a, SomeType b);
```

і:

```
template <class SomeType>
SomeType GetMax (SomeType a, SomeType b);
```

повністю еквівалентні.

Хоча typename явно краще відображає суть, існує широка практика використовувати class. Як виправдовується Страуструп[1]
"Використовую class, заспокоюючи себе, що це на кілька літер менше набирати".

В цих лекціях я намагатимуся використовувати typename, але будуть винятки.

[1] Не можу знайти посилання, тому в моєму переказі, із всіма помилками на моїй совісті.

Складніші випадки виведення типів

У випадку виникнення неоднозначності, маємо помилку:

```
cout << GetMax(a2,b1) << endl;
```

```
error C2782: 'SomeType GetMax(const SomeType &,
    const SomeType &)' : template parameter 'SomeType'
    is ambiguous
see declaration of 'GetMax'
                could be 'double'
                or       'int'
```

Виходів є декілька:

- Явно вказати тип параметрів:

  cout << GetMax<**double**>(a2,b1) << endl;

- Виконати явне приведення типів для одного із аргументів:

  cout << GetMax(**static_cast**<**double**>(a2), b1) << endl;

- **Дозволити аргументам мати різні типи**:

```
template <typename SomeType1, typename SomeType2>
SomeType1 GetMax (SomeType1 a, SomeType2 b)
{
    return (a>b?a:b);
}
```

Увага! Функція вище поводиться вульгарно — вважає, що завжди повертається тип першого переданого значення. На жаль, в C++-98/03 немає простого способу коректно вибрати тип значення, що повертається. В новому стандарті це виправлено.

Попередня функція з використанням C++11

```
template <typename T1, typename T2>
auto GetMax2_cpp11 (T1 a, T2 b) -> decltype( a>b ? a : b )
{
    return (a>b ? a : b);
}
```

Попередня функція в рамках **C++98**

Аналогічний код для C++98/03 буде значно більш громіздким. В цьому випадку теж можна зробити так, щоб компілятор вибирав більш "потужний" тип, однак написання його вимагає професійного володіння шаблонами, а отриманий код буде доволі важким для сприйняття.

Файл заголовків `cpp98_promote.h`, із прикладу `l17_function_templates`, який реалізовує цей функціонал, має більше сотні рядків.
Правда, результат такий же простий у використанні:

```
template <typename T1, typename T2>
TRES(T1,T2) GetMax2_cpp98 (T1 a, T2 b)
{
    return (a>b?a:b);
}
```

Шаблон класу — приклад

```
template <typename SomeType>
class StupidStack{
private:
    SomeType *data;
    size_t stack_size;
    size_t top;
public:
    StupidStack(size_t st_size): stack_size(st_size), top(0) {
        data = new SomeType[stack_size];
    }
    ~StupidStack(){ delete[] data; }
        .....................
    void push(const SomeType& in);
    SomeType top() const;
    void pop();
    inline bool empty() const { return top==0; }
    inline size_t size() const { return top; }
};

StupidStack<int> s1(20);
StupidStack<double> s2(20);
```

- Клас, що генерується із шаблона є звичайним, повноцінним, класом.
- Методи та поля шаблону оголошуються і визначаються абсолютно так само, як і для звичайного класу. Зокрема, їх можна визначати за межами шаблону:

```
template <typename SomeType>
    void OutStack<SomeType>::push(const SomeType& in) {
        ..........
}
```

- Всередині шаблону його іменем можна користуватися без конкретизації типу – вважається, що там "просто" OutStack означає поточну інстанціацію[2], хоча вона і дозволена:

```
template <typename SomeType> class OutStack{
        ..........
    OutStack* clone();
};
template <typename SomeType>
OutStack<SomeType>* OutStack<SomeType>::clone(){
    OutStack *res = .... // Можна і OutStack<SomeType> *res
    return res;
}
```

[2] Це називається "Injected class names", приклади див

- Тип, переданий шаблону, повинен реалізовувати інтерфейс, на який шаблон очікує. Скажімо, ось таке використання:

```
complex<double> c1{1,2}, c2{3,5};
complex<double> mc = GetMax(c1,c2);
```

призведе до помилки компіляції, так як `operator<` для комплексних чисел не визначений.

- Однак, якщо метод класу, котрому потрібен засіб, якого немає у типу-параметру, не використовується — помилки не буде!

```
template <typename SomeType>
class OutStack{
  .......
    int CheckElemLength()
    {
        return data[top-1].length();
    }
};
```

OutStack<**int**> T; *// OK*, поки не викликається *CheckElemLength*

Методи, що не використовуються, не інстанціюються.

Технічні подробиці

- Різні інстанціювання одного шаблона із тими ж параметрами вважається тим самим типом. Використання typedef-синонімів та константних виразів не змінює цього факту.
- Компілятор повинен бачити визначення шаблонів, тому їх слід включати в кожну одиницю трансляції. Немає простих способів надати скомпільовану реалізацію, як для звичайних класів та функцій[3].
- Під час оголошення шаблону відбуваються лише базові синтаксичні перевірки — ті, які можна зробити, не знаючи конкретних його параметрів. Повноцінна перевірка здійснюється під час інстанціювання.

[3] В теорії, деякі методи існують, але вони поки або не підтримуються, або ненадійні, або і те і інше. Зокрема, див. історію ключового слова export, визнаного застарілим в C++11.

Параметри шаблонів

Параметрами шаблонів можуть бути:

- Типи. Задаються як "`class T`" або "`typename T`". class і typename — рівноправні, хоча краще використовувати typename.
- Параметри не типи. Детальніше про них – далі.
- Параметри, які самі є шаблонами.
- Можливим є використання значень параметрів за замовчуванням.

```
template <class T1,          // параметртип−
          typename T2,       // параметртип−
          int I,             // параметр, який не є типом
          T1 DefaultValue,   // параметр не тип
          template <class> class T3, // параметршаблон−
          template <typename> class T3a, // параметршаблон−
          class Character = char // параметр із значенням
                                 //за замовчуванням
         >
```

Детальніше див., крім книг Страуструпа та Вандервурда, наприклад:
`http://en.cppreference.com/w/cpp/language/template_parameters`

Параметри шаблонів – не типи

На такі параметри накладаються певні обмеження. Вони можуть бути:

- Константами інтегральних типів[4] чи перерахування (enum).
- Вказівниками чи посиланнями[5] на об'єкти із статичним часом існування[6], як із зовнішнім, так і з внутрішнім зв'язуванням[7] та функції із зв'язуванням (linkage)[8].
- Вказівники на члени, виду `&Class::Member`.
- `std::nullptr_t`.
- C++20 дозволив floating point та типи, які можуть бути результатом constexpr функції (literal types) із всіма публічними незмінними членами.
- ◇ Приклад використання (кастомних) стрічок як аргументів: `https://vector-of-bool.github.io/2021/10/22/string-templates.html`.

[4] Тобто, floating-point і об'єкти типу класів заборонені.

[5] В світлі стандарту C++11 – lvalue-посилання.

[6] Тобто, такі, що існують від початку програми до її завершення. Англійською – static storage duration.

[7] Дещо спрощуючи, об'єкти із зовнішнім зв'язуванням доступні із різних одиниць трансляції, а з внутрішнім – лише із своєї. Див. також http://www.goldsborough.me/c/c++/linker/2016/03/30/19-34-25-internal_and_external_linkage_in_c++/.

[8] Сутності без зв'язування – оголошені у локальних областях видимості, зокрема – змінні без extern, локальні класи тощо. Див. також http://en.cppreference.com/w/cpp/language/storage_duration#Linkage.

- Увага! Текстові константи мають внутрішнє зв'язування – навіть коли вони мають одне значення, у різних одиницях трансляції це можуть бути різні об'єкти. Тому їх використовувати у ролі параметрів шаблонів не можна – це було визнано занадто проблемним[9], бо інстанціювання із однаковим аргументом із різних одиниць трансляції могли б ставати різними типами[10]. Але так робити можна:

  template <**const char*** str> **class** X { .... };
  extern char const text[]="ABC";
  X<text> x;

- Увага! Вказівники на елементи масиву використовувати не можна – елементи не мають зв'язності. Наступний код невірний:

  X<&text[1]> bad; // Буде помилка компіляції

- Те ж стосується нестатичних членів класів.

[9] Див, наприклад: https://stackoverflow.com/questions/5547852/string-literals-not-allowed-as-non-type-template-parameters.

[10] Але див. http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2017/p0424r2.pdf.

Вказівники на об'єкти та функції як аргументи шаблонів

Синтаксис дещо незвичний, тому наведемо приклад:

```
template<int N>
struct Data {
    int d1[N];
    int d2[N];
};
template<
    double *dp,     // Вказівник на обєкт'
    int (&ra)[5],   // lvalue посилання — на масив
    int (*pf)(int), // вказівник на функцію
    int (Data<10>::*a)[10] // вказівник на член класу, типу int[10] >
class Y {};

int fn(int x) { return x; }
double m;
int arr[5];
    .....
    Y<&m, arr, fn, &Data<10>::d1> obj;
```

Тобто, вказівник, що передається класу, повинен мати вигляд &of якщо of — ім'я об'єкта або функції, або в формі f, якщо f — ім'я функції чи масиву.

Деталі шаблонів функцій

- Якщо визначення типів аргументі функції не справляється, можна їх явно конкретизувати. Конкретизація всіх параметрів не є обов'язковою, однак вказувати їх можна лише послідовно, зліва направо:

```
template<typename T, typename U> T implicit_cast(U u) { return u;}

int i = ...;
implicit_cast(i); // Помилка! Немає способу визначити
                  // тип, до якого приводити

implicit_cast<double>(i); // T -- double, U – int

implicit_cast<char, double>(i); // T -- char, U – double

implicit_cast<char*, int>(i); // Помилка! Не можна
                              // неявно перетворити int в char*
```

Перевантаження шаблонів функцій

- Можна оголошувати декілька шаблонів функцій, із тим же іменем, але різними наборами параметрів[11]. Крім того, можуть існувати і звичайні – не шаблонні, функції з цими ж іменами.
- Правила перевантаження при цьому є узагальненням правил перевантаження звичайних функцій: для кожного шаблону здійснюється підбір найкращої спеціалізації згідно списку аргументів функції. Далі до отриманого списку "конкретних" перевантажених функцій застосовуються майже звичайні правила перевантаження.
- Головне доповнення до цих правил:

1. Якщо є два однаково хороших співпадіння шаблонів, вибирається більш спеціалізований.
2. Зокрема, конкретна функція завжди вибиратиметься замість такої ж хорошої, але шаблонної.

- Увага! Забігаючи наперед, спеціалізації функцій не беруть участі в перевантаженні – лише вихідні шаблони! Детальніше див. "Why Not Specialize Function Templates?" Герба Саттера. Тому їх варто уникати[12].

[11] Увага! З класами так робити не можна!

[12] Ще приклад: "Function overloading is more flexible (and more convenient) than template function specialization".

Параметри за замовчуванням

```
template <typename T> class SorterLess{
public:
    bool operator()(T a, T b) { return a<b;      };
};
template <typename T> class SorterGreater{
public:
    bool operator()(T a, T b) { return a<b; };
};
template <typename T, class Sorter = SorterLess<T> >
class SortedContainer {
        vector<T> data;
public:
    typedef T DataType;
      ......
};
  ...........
    SortedContainer<int > as;
    SortedContainer<int, SorterLess<int> > as1;
    SortedContainer<int, SorterGreater<int> > as2;
```

Правило конкретизації те ж, що і для функцій — зліва направо, без пропусків

Спеціалізація

- Іноді є потреба вказати конкретну, відмінну від загальної, реалізацію для певних типів або класів типів. Наприклад — для стрічок окрема реалізація. Або для вказівників — окрема реалізація. Або, якщо тип не задовольняє певним вимогам — повідомляти про помилку.
- Приклад, масив елементів довільного типу:

```
template <typename T>
class VectorT{
  ..........
};
```

Ми вирішили зберігати булівські змінні більш ефективно, скажімо, в окремих бітах:

```
template<>
class VectorT<bool>{
  .........
};
```

Використання:

```
VectorT<int> t1;
VectorT<bool> t2;
```

- Існує повна спеціалізація і для шаблонів функцій:

```
template <typename T>
T GetMax (T a, T b) { return (a>b?a:b); }
template <>
// const char* GetMax<const char*> (const char* a, const char* b) {
const char* GetMax(const char* a, const char* b) {
    return ( strcmp(a,b)>0 ? a:b);
}
char ac[]="aaa", bc[]="bbb";
cout << GetMax(ac,bc) << endl;
```

- Однак, хоча для шаблонів класів спеціалізація є надзвичайно потужним інструментом, для шаблонів функцій в більшості випадків краще її уникати.
- Замість спеціалізації можна скористатися перевантаженням:

```
const char* GetMax(const char* a, const char* b){  .....  }
```

- Справа в тому, що спеціалізації **не беруть участі в перевантаженні**! Перевантаження вибирає базовий шаблон, а інстанціація застосовується лише після, якщо його було вибрано.
- Детальніше див. "Why Not Specialize Function Templates?" від Герба Саттера.
- "Template Best Practices" – ще один цікавий есей.

Часткова спеціалізація

Нехай ми вирішили вказівники опрацьовувати окремо. Для цього можна написати **часткову** спеціалізацію, такі що конкретизує лише деякі властивості типів:

```
template<typename T>
class VectorT<T*>
{
  ..........
};
```

- Не можна створювати часткові спеціалізації шаблонів функцій.
- Загальний шаблон повинен йти перед будь-якою його спеціалізацією. Всі спеціалізації повинні бути в тому самому просторі імен, що і сам шаблон.

Успадкування і шаблони

- Публічне успадкування шаблону від нешаблонного класу – зручний метод сумістити зручність поліформізму із потужністю шаблонів.
- Приватне успадкування шаблону від нешаблонного класу — можливість заховати небезпечну реалізацію, не допускаючи роздування коду.
- Приклад:

```
class OrdinaryBase {};

template<typename T>
class TemplDerived: public OrdinaryBase { };
```

- Можливим є успадкування від іншого шаблону:

```
template <typename T>
class TemplatedBase {};

template<typename T>
class TemplDerived2: public TemplatedBase<T> { };

class NonTemplDerived: public TemplatedBase<int>
{ };
```

- Ще одне використання – запобігання роздуванню шаблонів, зменшення часу компіляції – код, незалежний від параметрів шаблонів, винести в базовий клас.
- Приклад такої трансформації – із такого:

```
template <typename T>
class my_vector {
    size_t m_size;
public:
    size_t size() const { return m_size; }
        ..................
};
```

- Робимо таке:

```
class my_vector_base {
protected:
    size_t m_size;
public:
    size_t size() const { return m_size; }
};

template <typename T>
class my_vector : public my_vector_base {
        ..............
```

- Спеціалізація ніяк не пов'язана з вихідним шаблоном.
- Однак, якщо у спеціалізації хотілося б скористатися напрацюваннями загального шаблону, це можна зробити, скориставшись успадкуванням.
- Приклад, як це можна зробити, у цьому есеї: "In a C++ class template specialization, how can I call the unspecialized version of a method?".

Успадкування і шаблони – подробиці

- Якщо успадкувати шаблон від шаблона, функції із предкового класу не можна буде викликати безпосередньо:

```
template <typename T>
class Base{
public:
    void fn(){    .....      }
};

template <typename T>
class Derived: public Base<T>{
public:
    void otherFn(){
        fn();    // <--- Error: there are no arguments to 'fn'
                 // that depend on a template parameter, so a
                 // declaration of 'fn' must be available (GCC)
    }
};
```

- Причина – виклик (безпосередньо) не залежить від T, тому компілятор не шукатиме функцію в залежному від T класі Base<T>.

- Це буде проблемою лише в методах такого шаблону – в нащадку-не шаблоні чи для об'єкту нащадка функція знаходитиметься успішно:

```
class Derivedl : public Base<T>{
public:
    void otherFn2(){
        fn();    // OK
    }
};
  .....
    Derived<int> d;

    d.fn();                 // OK
```

- Виходів є три. Перших два – використання this, що зробить ім'я залежним від аргументу шаблону або явна кваліфікація.

```
template <typename T>
class Derived: public Base<T>{
public:
    void otherFn(){
        this->fn();
        Base<T>::fn();
    }
};
```

- Третій – явне оголошення функцій з предка в нащадку:

```
template <typename T>
class Derived: public Base<T>{
public:
    using Base<T>::fn;
    void otherFn(){
        fn();
    }
};
```

- Див. також "Why can't I find the injected name of a templated class's templated base class?"

Члени-шаблони

- Методи класу (як нешаблонного так і класу-шаблону) можуть бути шаблонами самі по собі.
- Наприклад, щоб дозволити перетворення між векторами із різними типами, можна написати ось так:

```
template <typename T>
class VectorT {
    T *data;
    size_t size;
public:
    .....
    T&       operator[](size_t i)       { return data[i]; }
    const T& operator[](size_t i) const{ return data[i]; }

    template <typename U>
    void copy_from(const VectorT<U>& in) {
        for(size_t i=0; i<size; ++i){
            data[i] = in[i];
        }
    }
};
```

- Тоді можна буде робити ось так:

```
VectorT<int> t1;
VectorT<double> t1d;
t1d.copy_from(t1);
```

- **Увага! Шаблонні конструктори копіювання та оператори присвоєння не є повноцінними конструкторами.** Тому, слід також надавати їх нешаблонні варіанти, інакше їх буде згенеровано за замовчуванням.
- Зокрема, якщо б ми замість методу `copy_from` вирішили б написати відповідний конструктор, то довелося б робити так:

```
template <typename T>
class VectorT {
        ........
public:
        ........
    template <typename U>
        VectorT(const VectorT<U>& in) { ..... }
    VectorT(const VectorT<T>& rhs)    { ..... }
};
```

- Визначення за межами оголошення класу:

```
template <typename T>
    template <typename U>
    void VectorT<T>::copy_from(const VectorT<U>& in) {
        for(size_t i=0; i<size; ++i){
            data[i] = in[i];
        }
    }
```

- Спеціалізацію шаблонного методу шаблону слід створювати **за межами оголошення класу**, однак, при тому, слід також повністю спеціалізувати сам клас, тобто, єдиний коректний варіант:

```
template <>
template <>
void VectorT<int>::copy_from<char>(const VectorT<char>& in){ ... }
```

- Зберегти параметризацію самого класу так не вдасться. Але, як і з функціями, замість спеціалізації методів варто скористатися їх перевантаженням.

- Те ж обмеження стосується і повної спеціалізації вкладених класів-шаблонів – її можна робити лише ззовні класу і лише повністю спеціалізуючи зовнішній клас.
- А от часткова спеціалізація можлива і в оголошенні класу.

```
template <typename T>
class VectorT {
    ......................
    template <typename X> class Test{};
    template <typename X> class Test<X*>{};
};

template <>
template <>
class VectorT<int>::Test<char>{};
```

- Більше екзотичних прикладів див. `http://en.cppreference.com/w/cpp/language/template_specialization` та `http://en.cppreference.com/w/cpp/language/partial_specialization`.

УВАГА! Методи-шаблони не можуть бути віртуальними!

Статичні члени даних та методи

- Шаблони можуть мати статичні дані та методи. Так як інстанційований шаблон – звичайний клас, кожен такий клас має **свою копію** статичних даних. Тобто, статичні дані відмінні для класів із різними наборами параметрів.
- Якщо є потреба, щоб всі інстанціації шаблону володіли однією копією статичних даних, слід помістити ці дані в нешаблонний базовий клас.
- Статичні члени слід визначати за межами оголошення шаблону.
- Дозволено використовувати спеціалізації.

Приклад статичних членів[13]

```
template <typename T>
class A {
        ........
    static T def_val;
    static T* some_fn(T a = def_val);
};

template <typename T>
    T A<T>::def_val(0,0);
template <typename T> T
    T* A<T>::some_fn(T a) { ..... };
```

Спеціалізація для int:

```
template <>
    int A<int>::def_val<int>=0;
template <typename T> T
    int* A<int>::some_fn(int a) { ..... };
```

[13] За книгою Страуструпа.

Друзі шаблонів

- Різні інстанціації шаблону **не є друзями**. Щоб зробити їх друзями:

```
template<typename T>
class A{
    template <typename U>
        friend class A;
};
```

Так само можна зробити другом довільний шаблон.

- "Універсальна" дружба з функціями:

```
class A{
    .....
    template<typename U>
        friend U some_fn(A<U>& a);
};

template<typename T>
T some_fn(A<T>& a) { .... }
```

Шаблонна функція some_fn для будь-якого типу параметру буде другом всіх інстанціацій шаблону A.

Щоб другом A<T> була тільки some_fn<T>, можна зробити так:

1. **template**<**typename** T>
   class A;

2. **template**<**typename** T>
   int some_fn(A<T>& a);

3. **template**<**typename** T>
   class A{
   .....
   friend **int** some_fn<T>(A& a);
   };

4. **template**<**typename** T>
   int some_fn(A<T>& a) { .... }

Справа в тому, що в **п.3** слід вказати конкретну функцію — some_fn<T>. Для цього клас повинен знати, що some_fn — шаблон. **П.2** повідомляє це, оголосивши функцію. Однак, шаблон функції повинен знати про шаблон класу A. Останній попередньо оголошено (forward declaration) в п. 1.

Увага! Виклад питання дружби шаблонів вище – **дуже** поверхневий. В ньому є багато деталей та темних місць, як в теорії (стандарті), так і на практиці. Детальніше див.:

- Герб Саттер, "Новые сложные задачи на C++", задача 8, "Дружественные шаблоны"
- Бьерн Страуструп, "Язык програмирования C++", спеціальне видання ("специальное издание"), (C++98), додаток B, "Технические подробности", B.13.2 "Шаблоны"-"Друзья". Видання четверте, "C++11" — глава 23.4.7, "Abstraction mechanisms"-"Templates"-"Friends".
- Дэвид Вандервурд, Николаи Джосаттис "Шаблоны C++. Справочник разработчика", глава 8.4, "Вглубь шаблонов"-"Друзья".
- Хороший приклад нюансів: `https://blog.panicsoftware.com/friends-and-where-to-find-them/`.

Шаблони як параметри шаблонів

- Параметрами шаблонів можуть бути шаблони класів[14].

```
template <typename T>
class  my_vector{....};

template <typename T>
class  my_deque{....};

template <typename T, template<typename> class C>
class  S {
    C<T> vals;
    C<T*> ptrs;
        ..............
};
 .....
    S<int, my_vector> V1; // В ролі контейнера: my_vector
    S<int, my_deque>  V2; // В ролі контейнера: my_deque
```

- Аргументи можуть бути тими ж, що у шаблонів:

```
template <typename T, template<typename, typename> class C> class S2;
template <typename T, template<typename, size_t> class C>  class S3;
```

- З C++17 замість class можна писати typename.

[14] Шаблони функцій заборонені в ролі параметрів шаблонів.

Шаблони як параметри шаблонів — стандартні контейнери

- На жаль, так передати std::vector чи std:deque, **до C++17**, не вдасться.
- ◇ Їхні прототипи містять параметри за замовчуванням.
- ◇ При чому, їх кількість строго стандартом не регламентується. Хоча, зазвичай вони виглядають так, після алокатора можуть йти додаткові аргументи:

  template < **class** T, **class** Alloc = allocator<T> > **class** vector;
  template < **class** T, **class** Alloc = allocator<T> > **class** deque;

- ◇ Тому такий код призведе до помилки компіляції:

  S<**int**, std :: vector> v; *// Error!*

  але і спроба явно задавати алокатор буде, в загальному випадку, проблемною.

Шаблони як параметри шаблонів — стандартні контейнери

- Для бібліотеки, яка їх оголошує **саме так**, S можна було б реалізувати наступним чином:

```
template <typename T,
    template<typename U,
             typename Alloc = std::allocator<U> class C>
class S2{
    .............
      C<T > vals;
      C<T*> ptrs;
};
    ..............
S2<int, std::vector> V1;
S2<int, std::deque>  V2;
```

  Однак, тоді перестануть працювати інстанціювання із простішими класами, такими як my_vector вище.

- Зауважте, що аргументи шаблонних аргументів шаблонів можуть значення за замовчуванням, як у прикладі вище: `typename Alloc = std::allocator<U>`.

Шаблони як параметри шаблонів — C++17

- C++17 вирішило проблему із аргументами шаблонів за замовчуванням для шаблонів, що передаються як аргументи. Тепер код:

  S<**int**, std :: vector> v; *// OK for C++17*

  успішно скомпілюється із очікуваним результатом – аргументи за замовчуванням буде просто підставлено.
- ◇ Clang 15 має проблему із таким кодом (GCC12 і MSVC2019 – ні), але якщо замість std::vector використати синонім – працює:

  template<**typename** T>
  using vector_o = std::vector<T>;
  S<**int**, vector_o> v;

- Ще одне, дрібне, покращення, тепер можна писати не тільки так:

  template <**typename** T, **template**<**typename**> **class** C>
  class S {};

  але й так, вживаючи слово typename і праворуч:

  template <**typename** T, **template**<**typename**> **typename** C>
  class S {};

typename

- Ключове слово typename має ще одне важливе використання.
- Розглянемо на прикладі `std::vector`. Тип ітераторів для нього оголошено в ньому самому:

```
template<typename T, .....>
class vector{
    typedef T* iterator; // Лише приклад — це не обовязково' вказівник
        .....
};
  .....
    vector<int> a;
    vector<int>::iterator i = a.begin();
```

- Однак, у такій ситуації:

```
template<typename V>
void some_fn(V &v) {
    V::iterator itr = v.begin();
}
```

компілятор не має жодного способу здогадатися, що V::iterator — ім'я типу, тому буде помилка компіляції.

- Щоб допомогти компілятору, зробимо ось так:

```
template<typename V>
void some_fn(V &v) {
    typename V::iterator itr = v.begin();
}
```

- Такі імена називаються залежними, так як залежать від параметра шаблону.
- Компілятор знатиме всю інформацію, щоб їх підставити, лише в місці інстанціювання. До того він навіть не знає – це буде тип, значення чи й шаблон.
- Тому він припускає, що це буде не тип. В даному випадку це припущення помилкове і компілятору доводиться підказувати – це тип.
- ◇ На наступному слайді приклад, коли доводиться підказувати, це – шаблон.
- Детальніше див. `http://en.cppreference.com/w/cpp/language/dependent_name` та `http://www.modernescpp.com/index.php/types-non-types-and-templates-as-template-parameters`.
- В C++20 трохи зменшили потребу в typename: `https://shafik.github.io/c++/2021/01/07/down-with-typename.html`; `https://www.cppstories.com/2022/less-typename-cpp20/`.
- ◇ Зокрема, він більше не потрібен там, де не може бути нічого, крім типів.

Кваліфікатор template

- Розглянемо схожу ситуацію:

```
template<typename T>
class  ComplexEx{
        ........
    template <typename B> B convert_to();
};
  ........
template<typename V>
void some_fn2(V &v)
{
    // double d = v.convert_to<int>() ; <--- error
    double d = v.template convert_to<int>() ;
}
```

- Компілятор, "знаходячись" в `some_fn2` не знає, що в V має бути шаблонний метод `convert_to`, тому `convert_to<int>` сприймається як "`convert_to` менше int", що є дурницею. Щоб допомогти компілятору, слід використати template у ролі кваліфікатора:

```
    double d = v.template convert_to<int>() ;
```

- В C++20 трохи зменшили потребу в такому використанню template.

SFINAE

- SFINAE – Substitution failure is not an error.
- Вибираючи функції для перевантаження, компілятор здійснює інстанціювання шаблонів (якщо такі є, звичайно), якщо при цьому стається помилка – компілятор просто викидає таке інстанціювання із списку можливих, не вважаючи це помилкою.
- Приклад із wiki:

```
struct Test {
    typedef int foo;
};

template <typename T>
void f(typename T::foo) {} // Визначення 1

template <typename T>
void f(T) {}               // Визначення 2

int main() {
    f<Test>(10); // Викликає 1
    f<int>(10);  // Викликає 2, хоча int::foo й не існує.
}
```

SFINAE – 2

- SFINAE широко використовується для всіляких фокусів метапрограмування.
- Типовий приклад – перевірити, чи є в класі певний typedef:

```
template <typename T> struct has_typedef_foobar {
    typedef char yes[1];  // типи yes і no мають
    typedef char no[2];   // різний розмір

    template <typename C>
    static yes& test(typename C::foobar*); // Є такий —— результат
                                           // має тип yes, розміру 1
    template <typename>
    static no& test (...);   // Всі інші ситуації —— no, розміром 2
    static const bool value = sizeof( test<T>(nullptr))
                              == sizeof(yes);
};

cout << has_typedef_foobar<int>::value;
```

- Детальніше див. книги Александреску, Вандервурда чи хоча б https://en.wikipedia.org/wiki/Substitution_failure_is_not

SFINAE – 3

- В C++11 – ще простіше:

```
#include <type_traits>

template <typename... Ts> using void_t = void;

template <typename T, typename = void>
struct has_typedef_foobar : std::false_type {};

template <typename T>
struct has_typedef_foobar<T, void_t<typename T::foobar>> :
                                    std::true_type {};
```

Явне інстанціювання

- Компілятор генерує конкретний клас із шаблону та виконує всі синтаксичні перевірки за першого його використання — інстанціювання. Це не завжди зручно. Тому існує можливість явно попросити компілятор інстанціювати функцію-шаблон чи клас-шаблон із вказаним набором аргументів.

```
template<typename T>
class A{
        .....
    template <typename U> U templ_method();
};
template<typename V>
void some_fn3(V &v) { .... }

template class A<double>;
template class A<double>::templ_method(int);
template void some_fn3<char>(char);
```

- Ключове слово template без символів <> вказує, що це запит інстанціювання, а не оголошення шаблону.
- **Увага!** За явного інстанціювання класів інстанціюються всі його методи.

- Поради щодо явної інстанціації із std: `https://quuxplusone.github.io/blog/2021/08/06/dont-explicitly-instantiate-std-templates/` – ніколи не робіть так.
- Нестандартний атрибут із GCC, `__attribute__((exclude_from_explicit_instantiation))` дозволяє виключити деякі методи із явної інстанціації – зокрема, якщо вони служать лише для реалізації.
- Як зробити аргумент шаблону функції обов'язковим, або, навпаки, як заборонити його явне використання: `https://quuxplusone.github.io/blog/2023/12/17/function-template-parameters/`.

export — deprecated

Щодо розділеної компіляції шаблонів. Експеримент із export виявився невдалим, і зараз воно вважається застарілим. Однак, кому цікаво, або потрібно, див.

- Дэвид Вандервурд, Николаи Джосаттис "Шаблоны C++. Справочник разработчика", глава 6.3, "Применение шаблонов на практике"-"Модель разделения".
- Герб Саттер, "Новые сложные задачи на C++", задачі 9 та 10, "Ограничения экспорта".
- Бьерн Страуструп, "Язык програмирования C++", спеціальне видання ("специальное издание"), (C++98), глава 13.7, "Шаблоны"-"Организация исходного кода". Видання четверте, "C++11" згадує про цю можливість лише "рудиментарно".

Java generics

- Java містить схожі засоби, узагальнені типи – Java generics. Однак, у них є принципова відмінність від шаблонів C++.
- Увага: автор цього тексту – не фахівець в Java. Можуть бути неточності, не всі нюанси озвучені, але він сподівається – сам принцип передано правильно.
- Приклад – аналог std::pair із C++:

```
public class Entry<KeyType, ValueType> {
    private final KeyType key;
    private final ValueType value;

    public Entry(KeyType key, ValueType value) {
        this.key = key;
        this.value = value;
    }
    public KeyType getKey() { return key; }
    public ValueType getValue() { return value; }
}

Entry<String, Integer> mark = new Entry<>("Test", 314);
```

- Однак, на відміну від інстанціювання в C++, яке є просто генерацією коду, в Java використовується той факт, що всі класи[15] є нащадками Object.
- Тобто, такий код:

```
public static <T> int count(T[] list, T item) {
    int count = 0;
    for ( T listItem : list )
        if (item.equals( listItem ))        count++;
    }
    return count;
}
```

перетворюється у щось таке:

```
public static <T> int count(Object[] list, Object item) {
    int count = 0;
    for ( T listItem : list )
        if (item.equals( listItem ))        count++;
    }
    return count;
}
```

[15] Крім 8 так званих примітивних типів, int і т.д.

- Лише для методів типу find(), які повертають об'єкт з відповідного контейнера, компілятор здійснює перетворення назад. Тобто, такий код:

```
class Rectangle extends Figure { ... }

class Finder{
        public static <T> T my_find count(T[] list, T item) { .... }
                .....................................
```

Rectangle r = my_find(figuresArray, myR);

перетвориться в (через <Object> – псевдокод):

Rectangle r = (Figure)my_find<Object>((Object[])figuresArray,

- Спеціалізація неможлива. Така конструкція всього лиш впливає на вибір компілятором типу результату:

Rectangle r = Finder.<Rectangle>my_find(figuresArray, myR);

- З іншого боку, те, що в C++ називається контрактним програмуванням та концепціями (C++20), в Java – давно.

"> >"

- Раніше, в C++98, наступний код був некоректним:

  vector<list<**double**>> x;

  Два > підряд сприймалися як оператор бітового зсуву, тому потрібно було писати:

  vector<list<**double**> > x;

  Тобто, розділяти два послідовних ">" пробілом.
- В C++11 пробіл перестав бути обов'язковим, і коректними будуть як перший, так і другий варіанти.
- Якщо в аргументі трапляється оператор зсуву — в обох стандартах його слід брати в дужки:

  vector< array<**double**, (n>>2)> > x;

Variadic templates

- C++98 не включала можливості створення типобезпечних функцій із змінною кількістю аргументів. Не було і шаблонів із змінною кількістю параметрів.
- Без першої із цих можливостей неможливо створити типобезпечний printf() і т.д. Без другої — створення кортежів (tuples) стає проблемою.
- Для вирішення цих труднощів у C++11 було додано шаблони із змінною кількістю аргументів — variadic templates.

Типовий приклад 1 — printf

- Спочатку реалізовуємо загальний варіант[16]:

```
template<typename T, typename... Args>
void ts_printf(const char* s, T value, Args... args){
    while (s && *s) {
        if (*s=='%' && *++s!='%') {
            std::cout << value;
            // Викликаємо printf() з урізаним списком аргументів
            return ts_printf(++s, args...);
        }
        std::cout << *s++;
    }
    throw std::runtime_error("Too many arguments provided for ts_printf");
}
```

- Вона, користуючись: `template<typename T, typename... Args>`, один аргумент опрацьовує сама, а решту рекурсивно передає сама собі.
- Для простоти, аргумент форматування ігнорується – потрібно лише, щоб він був.

[16] Реалізацію, із дрібними змінами взято тут: https://isocpp.org/wiki/faq/cpp11-language-templates#variadic-templates.

- Залишається опрацювати варіант без аргументів, який виводить лише стрічку формату:

```
void ts_printf(const char* s){
    while (s && *s) {
        if (*s=='%' && *++s!='%')
            throw runtime_error("Invalid format: missing arguments");
        std::cout << *s++; // Просто виводимо символи
    }
}
```

- Використання:

```
ts_printf("This is text, with %%, but without data.\n\n");
string s1="THIS IS STRING";
char   cs2[]="this is c-string";
int x = 5;
double z = 1.23;
ts_printf("std::string printed: \"%s\",\nOld-style string: \"%s\""
          ",\nint: %s,\ndouble: %s.\n",
                s1, cs2, x, z);
```

Типовий приклад 2 — tuple

- Кортеж можна створити теж скориставшись рекурсією[17]:

```
template<typename ... Types> class my_tuple;

template<> class my_tuple<> {};

template<typename Head, typename ... Tail>
class my_tuple<Head, Tail...>: private my_tuple<Tail...>{
    Head m_head{ Head() };
public:
    my_tuple(){};
    my_tuple(Head const& f, Tail const& ... tail ):
        my_tuple<Tail...>(tail ...), m_head(f)
    {}

    Head const& head() const { return m_head; }

    my_tuple<Tail...> const& tail() const { return *this; }
};
```

- При тому структура об'єктів буде такою:

```
class my_tuple<int, double, char>: private my_tuple<double, char>;
class my_tuple<double, char>: private my_tuple<char>
class my_tuple<char>: private my_tuple<>
```

[17] Взято із змінами тут: http://www.devx.com/cplus/Article/41533/0/page/3. Див. також дещо більш елегантну

- Використовувати її можна так:

```
my_tuple<> t0;
my_tuple<int> t1(5), t1a;
my_tuple<double, int> t2(3.14, 11);
my_tuple<string, double, int> t3("aaa",  1.5,  2);

cout << t1.head() << endl;
cout << t1a.head() << endl;
cout << endl;

cout << t2.head() << ", ";
cout << t2.tail().head() << endl;
cout << endl;

cout << t3.head() << ", ";
cout << t3.tail().head() << ", ";
cout << t3.tail(). tail().head() << endl;
cout << endl;
```

- Однак, це не дуже зручно.

- Для більш повноцінної реалізації нам знадобиться допоміжна структура, my_tuple_entry:

```
template<unsigned index,typename ... Types>
struct my_tuple_entry;

template<typename Head, typename ... Types>
struct my_tuple_entry<0, Head, Types...> {
    typedef Head const& type;

    static type value(my_tuple<Head, Types...> const& t) {
        return t.head();
    }
};

template<unsigned index, typename Head, typename ... Types>
struct my_tuple_entry<index, Head, Types...> {
    typedef typename my_tuple_entry<index-1, Types...>::type type;

    static type value(my_tuple<Head, Types...> const& t){
        return my_tuple_entry<index-1, Types...>::value(t.tail());
    }
};
```

- Щоб скористатися виведенням типів, додамо ще функцію:

```
template<unsigned index,typename ... Types>
typename my_tuple_entry<index,Types...>::type
get_tuple_entry(my_tuple<Types...> const& t) {
    return my_tuple_entry<index,Types...>::value(t);
}
```

- Тоді можна писати:

```
my_tuple<> t0;
my_tuple<int> t1(5), t1a;
my_tuple<double, int> t2(3.14, 11);
my_tuple<string, double, int> t3("aaa",  1.5,  2);

cout << get_tuple_entry<0>(t1) << endl;
cout << endl;

cout << get_tuple_entry<0>(t2) << ", ";
cout << get_tuple_entry<1>(t2) << endl;
cout << endl;

cout << get_tuple_entry<0>(t3) << ", ";
cout << get_tuple_entry<1>(t3) << ", ";
cout << get_tuple_entry<2>(t3) << endl << endl;
```

- Залишається скористатися автоматичним виведенням для створення кортежів:

```
template<typename ... Types>
my_tuple<Types...> make_my_tuple(Types... args){
    return my_tuple<Types...>(args...);
}
```

- Щоб можна було робити так:

```
auto mt = make_my_tuple(5, 3.14, "abc");
```

Технічні подробиці шаблонів із змінною кількістю аргументів

- Розглянемо фрагмент — шаблон функції із змінною кількістю аргументів:

```
template<typename ... Types>
void fn(Types ... args){ .....};
```

- `typename... Types` вказує, що Types це пакет параметрів шаблону (template parameter pack).
- `Types... args` вказує, що args — пакет параметрів функції (function parameter pack).
- Три крапки (ellipsis), означають "нуль або більше елементів".
- Пробіли можуть бути присутні чи відсутні із обох сторін від "...".
- Можна розгорнути пакет параметрів у послідовність елементів, поставивши три крапки після нього:

```
template<typename T, typename... Args>
void ts_printf(const char* s, T value, Args ... args){
        ..................
            return ts_printf(++s, args ...);
        ..................
}
```

- Можна розгортати пакет так, щоб для кожного елемента було викликано, скажімо, якусь функцію. Наприклад, узагальнення розглянутого раніше декоратора:

```
template<typename Callable, typename ... Types>
auto vdecorator( Callable  c,  Types&&... args) {
        -> decltype(c(forward<Types>(args)...))
    cout << "Starting function" << endl;
    return c(forward<Types>(args)...);
}

void fn1(int x){    .....    }

void fn2(int x, int y){    .....    }

void fn3(int x, int y, int z){    .....    }

    vdecorator(fn1, 5);
    vdecorator(fn2, 22, 33);
    vdecorator(fn3, 44, 555, 7777);
```

- Зауважте, що відбувається одночасне розпаковування двох пакетів — Types і args. Кількість їх елементів має співпадати.

- Цей механізм працює наступним чином: все, що робиться над пакетом аргументів, повторюється для кожного його елемента, тому можна робити навіть так:

```
template<typename ... Types>
void fn(Types ... args){
    ....................
    fn_for_ptrs(&args ...);
    ....................
}
```

- Якщо розгортання пакету вкладене у інше його розгортання, спершу відбувається внутрішнє, а потім зовнішнє, із повторенням кожного разу результатів розпакування внутрішнього:

```
template< typename T >
void print_args(const T& e){    cout << e << endl;  }
template<typename T, typename ... Types>
void print_args(T arg, Types ... args) {
    cout << arg << ", ";    print_args(args ...);
}

template< typename T >  T add(const T& e) {    return e;   }
template<typename T, typename ... Types>
T add(T arg, Types ... args) {
    return arg+add(args ...);
}

template<typename ... Types> void exotic(Types ...args){
    print_args(add(args...)+" " + args ...);
}

exotic( string{"a"}, string{"b"}, string{"c"});
// abc a, abc b, abc c
```

- Якщо два пакети аргументів трапилися у тому ж виразі, вони розгортаються поелементно, кількість елементів має співпадати. Один приклад був вище, тут ще один[18]:

```
template<typename...> struct Tuple {};
template<typename T1, typename T2> struct Pair {};

template<class ...Args1> struct zip {
    template<class ...Args2> struct with {
        typedef Tuple<Pair<Args1, Args2>...> type;
    };
};

typedef zip<short, int>::with<unsigned short, unsigned>::type T1;
// Pair<Args1, Args2>... розгортається в
// Pair<short, unsigned short>, Pair<int, unsigned int>
// Тоді T1 — Tuple<Pair<short, unsigned short>, Pair<int, unsigned>>

// typedef zip<short>::with<unsigned short, unsigned>::type T2;
// Помилка ——— різна довжина!
```

[18] Приклад з http://en.cppreference.com/w/cpp/language/parameter_pack.

Використання "..." — 1

Розгортання пакету параметрів може відбуватися в більшості місць, де може бути список[19]:

- У виклику функції — розглянуто вище:

```
f(&args ...);  // expands to f(&E1, &E2, &E3)
f(n, ++args...); // expands to f(n, ++E1, ++E2, ++E3);
f(++args..., n); // expands to f(++E1, ++E2, ++E3, n);
f(const_cast<const Args*>(&args)...);
// f(const_cast<const E1*>(&X1), const_cast<const E2*>(&X2), const_
f(h(args ...) + args ...); // expands to
// f(h(E1,E2,E3) + E1, h(E1,E2,E3) + E2, h(E1,E2,E3) + E3)
```

- У виклику конструкторів за різних видів ініціалізації:

```
Class c1(&args ...);                  // calls Class::Class(&E1, &E2, &E3)
Class c2 = Class(n, ++args...); // calls Class::Class(n, ++E1, ++E2,
```

[19] Частина прикладів звідси: http://en.cppreference.com/w/cpp/language/parameter_pack.

Використання "..." — 2

- В списку аргументів функції — розглянуто вище:

```
template<typename ...Ts> void f(Ts...) {}
f('a', 1);   // Ts... expands to void f(char, int)
f(0.1);      // Ts... expands to void f(double)

template<typename ...Ts, int... N> void g(Ts (&...arr)[N]) {}
int n[1];
g<const char, int>("a", n); // Ts (&... arr)[N] expands to
                            // const char (&)[2], int (&)[1]
```

- В списку аргументів шаблону під час інстанціювання:

```
template<typename T, typename ...Args>
struct Ex {
    tuple<T, Args...> t1;
    tuple<Args..., T> t2;
};
```

Використання "..." — 3

- В тому числі і як аргументи-не типи[20]:

```
template<typename... T> struct value_holder
{
    template<T... Values>
    struct apply { };
};
```

- В списку базових класів та в списку ініціалізації членів класу:

```
template<class... Mixins>
class X : public Mixins ... {
 public:
    X(const Mixins&... mixins) : Mixins(mixins )... { }
};
```

[20] Приклад звідси: http://en.cppreference.com/w/cpp/language/parameter_pack.

Використання "..." — 4

- В списках ініціалізації:

```
template<typename... Types> void fn(Types... args){
    const int size = sizeof ...( args ) + 2;
    int res[ size ] = {1,args ...,2};

    // Так як списки ініціалізації гарантують порядок, цим можна
    // скористатися для виклику функції для кожного аргументу
    // по порядку:
    int dummy[sizeof...(Ts)] = { (std::cout << args, 0)... };
}
```

- В списках захоплення лямбда-функцій:

```
template<class ...Args> void f(Args ... args) {
    auto lm = [&, args ...] { return g(args ...); };
    lm();
}
```

Використання "..." — 5

- В операторі `sizeof...`, який повертає кількість елементів у пакеті і теж, фактично, є розпакуванням.
- І т.д. – в динамічній специфікації виключень — throw(Args...), вказанні вирівнювання чи атрибутів.
- В списках шаблонних аргументів шаблонів:

```
template <template <typename, typename...> class ContainerType,
          typename ValueType, typename... Args>
void print_container(const ContainerType<ValueType, Args...>& c) {
  for (const auto& v : c) {
    std::cout << v << ' ';
  }
  std::cout << '\n';
}
```

Для map потребуватиме ще оператор виводу для std::pair.
Приклад взято тут: `https://eli.thegreenplace.net/2014/variadic-templates-in-c/`.

Використання "..." — 6

- C++17 додав ще using-оголошення:

```
template <typename... bases>
struct X : bases ... {
        using bases::g ...;
};
X<B, D> x; // OK: B::g and D::g introduced
```

- C++-20 додав обмеження (constraints) з допомогою концепцій. Про них – далі.
- Детальніше див. `http://en.cppreference.com/w/cpp/language/parameter_pack`.

Оператор sizeof в C++11

1. Можна застосовувати sizeof до членів структур та класів, не створюючи їх екземплярів: `sizeof(C::field)`.
2. Для шаблонів із змінною кількістю аргументів (variadic templates) з'явився його варіант, що повертає кількість аргументів[21]:

```
template <typename... Args>
void print_size(Args... args)
{
  cout << sizeof...(args) << endl;
}
```

[21] Приклад взято за посиланням вище

Розташування пакетів аргументів

- Для класів пакет аргументів шаблону має бути останнім в списку.
- Для шаблонів функцій пакет може бути раніше, якщо всі аргументи після нього можуть бути виведені або мають значення за замовчуванням. Згадуючи розглянутий вище декоратор:

```
template<typename ... Types, typename Callable, ostream& os=cout>
auto vdecorator2( Callable  c,  Types&&... args)
        -> decltype(c(forward<Types>(args)...)) {
    os << "Starting function" << endl;
    return c(forward<Types>(args)...);
    os << "Finished function" << endl;
}
```

Посилання

Детальніше, крім FAQ та книги Страуструпа, див:

- "Cppreference: Parameter pack".
- IBM Knowledge Center, "Variadic templates (C++11)".
- "Pulling a single item from a C++ parameter pack by its index".

C++17 – fold expression

- Робота із пакетами аргументів регулярно вимагає використання рекурсії. Це часто не зручно.
- C++17 додав fold expression (згортку виразів[22] – термін із функціонального програмування), інструмент, який сильно спрощує виконання таких задач.
- Класичний приклад – функція, що знаходить суму своїх аргументів, скільки б їх не було. C++14, рекурсія:

```
auto sum_cpp14(){
    return 0;
}
template<typename T, typename... Types>
auto sum_cpp14(T s, Types... args){
    return s + sum_cpp14(args...);
}
```

- C++17, із тим же результатом:

```
template<typename ...Types>
auto sum_cpp17(Types ...args){
    return (args + ...);
}
```

[22] Не впевнений в перекладі, див. також https://en.wikipedia.org/wiki/Fold_(higher-order_function).

- Іноді пишуть так:

```
template<typename ...Types>
auto sum_cpp17(Types ...args){
    return (args + ... + 0);
}
```

але, як на мене, це занадто конкретно. Можна ще так:

```
template<typename Tinit, typename ...Args>
auto sum_cpp17(Tinit init, Args ... args){
    return (args + ... + init );
}
```

Але сенсу особливо немає. Це той же вираз, що і раніше.

- Використання, в будь якому із випадків:

```
std::cout << sum_cpp17(1,2,3,4) << std::endl;
std::cout << sum_cpp17(1) << std::endl;
std::cout << sum_cpp17(std::string("xy"),"234") << std::endl;
```

Сподіваюся, зрозуміло, чому без хоча б одного std::string (чи string literal) останній приклад не функціюватиме?

- Ще приклади:

```
template<typename ...Types>
void print_all(Types&&... args) {
    std::cout << __PRETTY_FUNCTION__ << std::endl;
    (std::cout << ... << std::forward<Types>(args)) << '\n';
}

template<template <typename> typename C, typename T,
                                          typename... Args>
void push_back_all(C<T>& v, Args&&... args){
    (v.push_back(forward<Args>(args)), ...);
}
```

- Виклик функції до кожного елемента контейнера:

```
(f(ts), ...);
```

як аналог:

```
for (auto elem : ts)
    f(elem);
```

- Цей і наступні приклади взяті із: `https://foonathan.net/2020/05/fold-tricks/`.

- Виклик функції до елементів контейнера у зворотному порядку:

```
int dummy;
(dummy = ... = (f(ts), 0));
// Буде розгорнуто в dummy = ((f(ts[0]), 0) = (f(ts[1]), 0)) = ...
// І через правоасоціативність— присвоєння, обчислено з права на ліво.
```

Це трохи брудний трюк, подробиці див. тут:
`https://quuxplusone.github.io/blog/2020/05/07/assignment-operator-fold-expression/`.

- Звідти дві рекомендації:
  - Use left folds only for left-associative operators.
  - Use binary right folds only for right-associative operators.
  - Use unary right folds only for right-associative operators, the short-circuiting logical operators, and comma.

- Викликати функцію для кожного елемента, поки предикат не поверне true.

  ((pred(ts) ? **false** : (f(ts), **true**)) && ...);

- Перевірити, чи хоча б один елемент задовольняє предикат:

  bool any_of = (pred(ts) || ...);

- Підрахувати, скільки таких елементів є:

  auto count = (std::size_t(0) + ... + (pred(ts) ? 1 : 0));

- Більше прикладів – за посиланням: `https://foonathan.net/2020/05/fold-tricks/`.

- Як воно працює? Нагадаємо код:

```
template<typename ...Types> auto sum_cpp17(Types ...args){
    return (args + ...);
}
std::cout << sum_cpp17(1,2,3,4) << std::endl;
```

Це так званий unary right fold, він еквівалентний (1+(2+(3+4))). Він унарний, оскільки не користується init(). Якщо б функція виглядала так:

```
template<typename ...Types> auto sum_cpp17(Types ...args){
    return (args + ... + 0);
}
```

це б був binary right fold. А якщо так:

```
template<typename ...Types> auto sum_cpp17(Types ...args){
    return ( ... + args );
}
```

то називався б unary left fold і був еквівалентний до (((1+2)+3)+4).

- Види згорток:

```
( pack op ... )            // unary right fold
      // (E op ...)  --> (E1 op (... op (E[N-1 op E[N] )))

( ... op pack )            // unary left fold
      // (... op E) --> (((E[1] op E[2]) op ...) op E[N])

( pack op ... op init ) // binary right fold
      // (E op ... op I) --> (E[1] op (... op (E[N-1] op (E[N] op I))))

( init op ... op pack ) // binary left fold
      // (I op ... op E) --> ((((I op E[1]) op E[2]) op ...) op E[N])
```

- op може бути будь-яким із: `+ - * / % ^ & | = < >`
  `<< >> += -= *= /= %= ^= &= |= <<= >>=`
  `== != <= >= && || , .* ->*` (32 штуки).
- Для бінарних згорток обидва оператори повинні співпадати.
- pack – пакет параметрів або вираз від нього.
- init – вираз без не розгорнутих пакетів.

- Якщо вираз із пакетом містить оператор із вищим пріоритетом, ніж op, слід скористатися дужками:

```
template<typename ...Args> int sum(Args&&... args) {
//     return (args + ... + 1 * 2); // Error: operator with precedence b
       return (args + ... + (1 * 2)); // OK
}
```

- Порожні пакети в унарних згортках дозволені лише для:

&& ——> value of empty pack is **true**
|| ——> value of empty pack is **false**
, ——> value of empty pack is **void**()

- Увага! Пакет із одним аргументом може давати несподівану поведінку (для > < == != <= >= && ||.). Наприклад: (5 && 7) матиме тип bool, а (5) – int:

```
template<typename ...Types> auto and_cpp17(Types ...args){
    return (args && ...);
}
    std::cout << and_cpp17(3,4,5);          // true
    std::cout << and_cpp17(7,0,11);     // false
    std::cout << and_cpp17(5);          // 5
    std::cout << and_cpp17(0);          // 0
    std::cout << and_cpp17();           // true
```

- Зауважте, що для операторів >, >=, <, <= отриманий вираз буде дещо дивним. Для присвоєння, в принципі, теж.
- Асоціативність операторів ігнорується, наприклад правоасоціативні оператори типу +=, лівою згорткою згортатимуться зліва направо.
- Детальніше див:
- `https://en.cppreference.com/w/cpp/language/fold`
- `http://cpptruths.blogspot.com/2016/12/understanding-fold-expressions.html`

using в ролі typedef

- Нагадаємо, що крім typedef, тепер у нас є зручніша форма створення синонімів – using. Рядки нижче – попарно тотожні:

```
typedef vector<double>::iterator  titr ;
using  titr  = vector<double>::iterator;

typedef double (*fnPtrT)(double, int );
using fnPtrT = double (double, int );
```

- Однак, using вміє багато більше!

Синоніми шаблонів

- using можна використовувати для створення синонімів шаблонів із вказанням частини параметрів:

```
template<typename T>
using vectored_stack = std::stack<T, std::vector<T>>;

template<size_t N>
using double_array = std::array<double, N>;

template<typename T, typename U>
using reverse_map = std::map<T, U, std::greater<T>>;

template<class T>
using ptr = T*;
```

```
vectored_stack<int>  vs;
double_array<5> da;
reverse_map<int, double> mrl;
ptr<int> pi;
```

- Однак, робити спеціалізацію таких синонімів так не можна!

```
template<typename T>
using vectored_stack = std::stack<T, std::vector<T>>;

template<typename T>
using vectored_stack = std::stack<T*, std::deque<T*>>; // ERR
```

  Звичайно, шаблонна магія (використання traits-класів) дозволяє досягати такого результату.
- Оголошені using шаблони є синонімами відповідних шаблонів — як і у звичному використанні using чи typedef.
- Завдяки цьому, спеціалізація вихідного шаблону коректно працюватиме і з синонімом – його використання обере відповідну спеціалізацію, якщо така існує.
- Детальніше див. `http://en.cppreference.com/w/cpp/language/type_alias`.

Шаблони змінних — C++14

- В C++14 з'явилася можливість оголошувати шаблони змінних:

```
// Enough digits for the 80-bit long double:
template<class T>
constexpr T pi = T(3.14159265358979323846264338);

template<class T>
T circular_area(T r) {
    return pi<T> * r * r;
}

// Або й так:
auto circular_area = [](auto r) { return pi<decltype(r)> * r * r; };
```

  pi<T> — інстанціювання змінної pi для типу T.
- Такі змінні дозволяють уникнути створення constexpr-функції лише щоб повертати константу.
- Ще одне важливе використання – заміна type traits:

```
template <typename T>
constexpr bool is_float = std::is_same_v<T, float>;
```

- Використовувати їх як аргументи шаблонів не можна.

Шаблони змінних — таки змінні

- const чи constexpr не є обов'язковим, хоча такий код виглядає, як мінімум дивним (і поки не зустрічав прикладів його використання):

```
// Enough digits for the 80-bit long double:
template<class T>
constexpr T pi = T(3.14159265358979323846264338);

    pi<int> = 5;
    cout << pi<int> << endl;    // Виведе 5
    cout << pi<double> << endl; // Виведе 3.14159
```

Шаблони змінних – членів класу

- Використання в класі створює статичні члени (зі всіма характерними особливостями статичних членів, аж по необхідність іноді окремо їх визначати).

```
template <typename T>
struct outer_traits {
    template <typename X>
    static constexpr bool sub_traits = std::is_same<T, X>::value;
};
struct limits {
    template<typename T>
    static const T min; // Оголошення
};
template<typename T>
const T limits::min = { }; // Визначення
```

- Ще приклади використання: `https://mariusbancila.ro/blog/2021/12/23/use-cases-of-variable-templates-in-cpp/` – спеціалізація для wide-стрічок і речі типу `is_floating_point_v`.

inline-змінні (C++20) (DRAFT)

- Власне, змінні, оголошені та визначені в хедері (порушуючи ODR), які, як і inline-функції, компілятор вважатиме однією змінною.
- Всі їх оголошення в різних одиницях трансляції мають співпадати – як і для inline-функцій.
- Приклад:

```
// constants.h -- enough digits for the 80-bit long double:
inline constexpr double pi { 3.14159265358979323846264338 };
```

- До C++23 (станом на червень 2021) додали константу Пі – нарешті!
- `https://habr.com/ru/company/otus/blog/561772/` – деякі подробиці.
- Потенційні проблеми inline-змінних із SFINAE: `https://www.cppstories.com/2022/sfinea-immediate-context/`.
- Приклад, коли їх можна відрізнити від не-inline: `https://quuxplusone.github.io/blog/2022/07/08/inline-constexpr/`.

Шаблони змінних – посилання

- Детальніше див: http://en.cppreference.com/w/cpp/language/variable_template,
- https://baptiste-wicht.com/posts/2017/08/simplify-your-type-traits-with-c%2B%2B14-variable-templates.html,
- та https://isocpp.org/wiki/faq/cpp14-language.

Локальні параметри шаблонів

- Тепер параметрами шаблонів можуть бути і локальні класи:

```
void fn(){
    class C{};

    vector<C> v;
}
```

- В C++98 такий код був заборонений, разом із відсутністю лямбда-функцій, різко ускладнюючи використання STL.

Extern template

- В C++98, компілятор інстанціює шаблон за кожного використання або у відповідь на запит явного інстанціювання. Якщо те саме інстанціювання трапляється в різних файлах багато раз, час компіляції жахливо зростає.
- В C++11 з'явилася можливість попросити компілятор **не інстанціювати** шаблон:

```
extern template class std :: vector<MyClass>;
```

template<auto> – C++17

- Тепер можна робити так:

```
template <auto value> void f() { }
template <auto value> constexpr auto constant_t = value;

f<10>();
constexpr auto const main_constant = constant_t <42>;
```

- Раніше доводилося:

```
template <typename Type, Type value> void f() { }
template <typename Type, Type value> constexpr Type constant_t


f<int, 10>();
constexpr auto const main_constant = constant_t<int, 100>;
```

template<auto> – C++17

- Це дуже спрощує, зокрема, створення гетерогенних списків констант часу компіляції:

```
template <auto ... vs> struct heterogenous_value_list {};
using  specific_list  = heterogenous_value_list<42, 7u, 13, '@'>;
```

Без цього їх створення стає значно довшим:

```
template <typename ... ValueTypes>
struct heterogenous_value_list_cpp14 {};
using specific_list_14 = heterogenous_value_list_cpp14<
        constant<int, 42>, constant<unsigned, 7u>, constant<int, 13>,
        constant<char, '@'> >;
```

- Однак, і гомогенні списки простіше створювати:

```
template <auto v0, decltype(v0) ... vs>
struct homogenous_value_list {};
using  specific_list2  = homogenous_value_list<1, 2, 3>;
```

замість:

```
template <typename T, T ... vs>
struct homogenous_value_list_cpp14 {};
using specific_list2_14 = homogenous_value_list_cpp14<int, 1, 2, 3>;
```

- Приклад із `https://stackoverflow.com/questions/48608830/template-template-parameter-of-unknown-type`:

```
template<std::size_t SIZE>
class Foo {};

template <template<auto> class T, auto K>
auto extractSize(const T<K>&) {
    return K;
}

int main() {
    Foo<13> f;
    std::cout << extractSize(f) << std::endl;
}
```

-

Template argument deduction for class templates (CTAD) – C++17

- Стандарт C++17 додав можливість виведення типу аргументів шаблону із виклику конструктора. Це дозволяє робити код менш багатослівним.
  - Стандартна бібліотека використовує ці можливості:

```
std::pair   p(2,  4.5);     // deduces std::pair<int,  double> p(2,  4.5);
std::tuple  t(4,  3,  2.5);
std::vector  vs{5,  std::string{}};
std::array  va{1,  2,  3,  4};          // va.size() == 4

std::mutex mtx;
auto lck  = std::lock_guard(mtx); // deduces: std::lock_guard<std::mute
```

- Працює навіть так – якщо конструктор дозволяє вивести всі необхідні аргумент шаблону – як типи, так і не-типи:

```
template<class T> struct A {
        A(T, T) {}
};
auto y = new A{1,2}; // A<int>
```

- Механізм – в цілому, той же, що і для виведення аргументів шаблонів функцій.

- Починаючи із C++20 може також виводити тип аргументу-не типу:

```
template<typename T>
struct X {
    constexpr X(T) {}
};

template<X x> struct Y {};
    ..............
Y<0> y; // OK, Y<X<int>(0)>
```

- Не працюють для оголошення не-статичних членів класу:

```
class SomeType {
    static inline std::vector v1 { 1, 2, 3}; // OK
                //  std::vector v2 { 1, 2, 3}; // Bad
};
```

- Також не працюватиме виведення кількості елементів:

```
class SomeType {
    // int arr1[] = {1, 2, 3};      // Bad
    // std::array arr2 { 1, 2, 3}; // Bad
};
```

Користувацькі CTAD

- Як на випадок не очевидних для компілятора правил виведення, так і для конструкторів, що є шаблонами, можна вказати користувацькі підказки – deduction guides.
- Приклад:

```
template <typename T>
class VectorT {
        .....................................
    using value_type = T;
    VectorT(const VectorT&);

    VectorT(const T& val, size_t copies);
    template <typename Iter> VectorT(Iter, Iter);
};
VectorT(const char*, size_t) -> VectorT<std::string>;
template <typename Iter> VectorT(Iter, Iter) ->
            VectorT<typename std::iterator_traits<Iter>::value_type>;
        .....................................
std :: vector<std::deque<int>> vdv;
VectorT vt1{vdv.begin(), vdv.end()};
VectorT vt2{"abc", 1};
static_assert(std::is_same_v<decltype(vt2)::value_type, std::string>);
```

- Застосовуються deduction guides лише якщо аргументи шаблону не вказані взагалі – часткового виведення не відбувається.
- Достатньо писати для типів – всілякі "декорації" типу l/r-valur reference, констант і т.д. не потрібні.
- Справа в тому, що deduction guides вибирають типи, для яких конструювати об'єкт, але не впливають на вибір перевантаження конструкторів.
- Не працюють з шаблонами-синонімами типів:

```
template <typename X>
using PairIntX = std::pair<int, X>;

PairIntX p{1, true}; // Помилка компіляції
```

- Додавання їх до класу – не інтрузивне, не впливає на інший код – не ламаючи сумісності.
- Для шаблонів конструкторів, при виборі перевантаження перевага надається підказкам deduction guides.
- Варто звертати увагу, що тепер дві змінні можуть виглядати як такі, що мають однаковий тип, маючи різний:

```
std::pair c(1, 2);
std::pair d(1, 2.0);
```

- explicite забороняє використовувати в copy-list-initialization:

```
template <typename Iter> explicit VectorT(Iter, Iter) ->
            VectorT<typename std::iterator_traits<Iter>::value_type>;

VectorT vt1{vdv.begin(), vdv.end()};      // OK
VectorT vt3 = {vdv.begin(), vdv.end()}; // Error
```

Виправлення в C++20

- Через те, що initializer_list-конструктори мають перевагу, в C++17 виникала неприємна ситуація – виводився не очевидний аргумент шаблону:

```
// C++17:
std :: vector  vt4{  std :: vector{1,  2,  3} };
                    // std :: vector<std:: vector<int>>,
                    // size ()  == 1
```

- В C++20 виправили, вимагаючи надавати перевагу конструктору копіювання, якщо ініціалізація відбувається одним елементом, який є спеціалізацією шаблону, що ініціалізується (або її нащадком):

```
// C++20:
std :: vector  vt4{  std :: vector{1,  2,  3} };
                    // std :: vector<int>,
                    // size ()  == 3
```

C++23

- C++23 додав підтримку CTAD для успадкованих конструкторів:

```
template<typename T>
class Base {
public:
    Base(T arg) {}
};

template<typename T>
class Derived : Base<T> {
public:
    using Base<T>::Base;
};

Derived d(42);  // Помилка в C++20, OK в C++23
```

Посилання

- https://devblogs.microsoft.com/cppblog/how-to-use-class-template-argument-deduction/
- https://medium.com/@barryrevzin/quirks-in-class-template-argument-deduction-ad08af42ebd6
- https://en.cppreference.com/w/cpp/language/class_template_argument_deduction
- https://arne-mertz.de/2017/06/class-template-argument-deduction/#User-defined_deduction_guides
- https://habr.com/ru/post/461963/ – "Class Template Argument Deduction".
- "Why isn't C++ using my default parameter to deduce a template type?".

Abbreviated function templates

- C++20 додав можливість записувати шаблони функцій скорочено[23]:

```
void fn1(auto arg) {    .....    }

void fn2(auto arg1, auto arg2) {    .....    }

size_t fn3(auto... arg1) { return sizeof...(arg1); }
```

- Також можна використовувати auto, обмежені концепціями:

```
void fn1(std::floating_point auto arg) {    .....    }
```

[23] Образно кажучи, спочатку auto будували на базі шаблонів, а тепер – auto повернулося назад.

Аргументи – не типи

- В C++20 до списку можливих аргументів шаблонів, що не є типами, додали:
  - Класи, що відповідають вимогам LiteralType – таким типам, які можна використовувати як constexpr змінними.
    - Якщо це класи-шаблони, їх аргументи можуть виводитися.
  - Floating point.
  - Лямбда-функції.

Демонстрація – об'єкти класів як аргументи шаблону

- Екземпляри таких об'єктів є const, mutable члени в них заборонені.
- Можна використовувати методи таких об'єктів.
- Поля можуть бути double, як і незалежні аргументи.

```
struct data_t{
    int x;                  double y;
    auto get_m() const { return x*y; }
};
template<data_t data>
void use_data(){
    cout << "y = " << data.y << "; r = " << data.get_m() << endl;
}
template<double coef>
auto get_val(double x){ return coef*x; }
    .....................
    constexpr data_t dt{2, 3.45};
    use_data<dt>();
    use_data<data_t{3, 4.56}>();
    cout << get_val<3.14>(10.0) << endl;
        // Не працює в Clang 15, GCC12 і MSVC2019 -- OK
```

Приклад виведення аргументів шаблону

```
template<std::size_t N>
struct fixed_string{
    constexpr fixed_string(const char (&s)[N+1]) {
        std::copy_n(s, N + 1, str);
    }
    constexpr const char* data() const {        return str;     }
    constexpr std::size_t size() const {        return N;       }
    char str[N+1];
};

template<std::size_t N>
fixed_string(const char (&)[N])->fixed_string<N-1>;

template<fixed_string S> // user-defined literals
constexpr auto operator""_cts(){    return S;   }

template<fixed_string S>  // N for 'S' will be deduced
void f(){
    std::cout << S.data() << ", " << S.size() << '\n';
}
    .....................
    f<"abc">(); // abc, 3
    constexpr auto s = "def"_cts;
    f<s>();     // def, 3
```

Посилання

- Приклад попереднього слайду взято звідси: `https://oleksandrkvl.github.io/2021/04/02/cpp-20-overview.html`.
  - Ймовірно, натхненний цим: `https://vector-of-bool.github.io/2021/10/22/string-templates.html`.
- "What's the deal with std::type_identity?" – про те, як заборонити/уникнути виведення типів для (деяких) використань типу-аргументу шаблону.

Індексування пакетів аргументів – C++26

- В C++26 запропоновано безпосереднє індексування пакетів аргументів.
- ◇ Одна з головних причин – прискорення компіляції.
- Приклад з документу з пропозицією, "Pack Indexing":

```
template <typename... T>
constexpr auto first_plus_last(T... values) -> T...[0] {
    return T...[0](values...[0] + values...[sizeof...(values)-1]);
}
```

- "C++26: pack indexing" – есей на тему.

Variadic-розпакування – C++26

- Планується (станом на 03-2024) прийняти в стандарт можливість робити так:

```
auto&& [...elems] = some_tuple;
```

- Приклади з документації:

```
template <class P, class Q>
auto dot_product(P p, Q q) {
    auto&& [...p_elems] = p;
    auto&& [...q_elems] = q;
    return (... + (p_elems * q_elems));
}
// Або:
template <class P, class Q>
auto dot_product(P p, Q q) {
    auto [... Is] = std::make_index_sequence<std::tuple_size_v<P>>{};
    return (... + (std::get<Is>(p) * std::get<Is>(q)));
}
```

- Працює лише в контексті шаблонів – не можна так робити за їх межами.
- Детальніше: "Structured Bindings can introduce a Pack".

Template for – C++26

- Додано цикли часу компіляції.
- Приклад з "P1306: Expansion Statements":

```
template <class... Ts>
void print_all(Ts... xs) {
  template for (auto elem : {xs...}) {
    std::println("{}", elem);
  }
}
```

- Зручний для роботи, поміж іншого, з tuple та рефлексіями.

- Як і раніше, інстанціації з однаковими аргументами не-типами, вважаються однаковими – тим же класом чи тією ж функцією
- Однак, для аргументів, які є об'єктами класів, порівняння здійснюється не їх оператором ==, а побітово.
  - Виняток – union, там ще відстежується активний член, і вважаються рівними, якщо активні члени ті ж, і їх значення рівні – побітово.

Концепції та шаблони зі змінною кількістю аргументів як аргументи шаблонів

- Приклади з документу:

```
template <template <typename T> concept C>
struct S {
  void f(C auto);
};

template <
  typename T,
  template <typename...> concept... C>
concept all_of = (C<T> && ...);

template <
  typename T,
  template <typename...> typename... U>
class TU {  .....  };
```

Інше

- Розпакування аргументів тепер можливе і для friend:

```
template <typename... T>
class D {
    friend T ...;
};
```

◇ Есей на тему: "C++26: variadic friends".

- "C++26: The Oxford variadic comma", виявляється, до C++26 в C++ можна було так:

```
void h(auto .....);    // Еквівалентно до (auto ..., ...)
```

- Трюк: створити фіксований тип, чи значення, який є залежним типом: https://devblogs.microsoft.com/oldnewthing/20200413-00/?p=103669.
- http://en.wikipedia.org/wiki/Template_metaprogramming http://en.wikibooks.org/wiki/C%2B%2B_Programming/Templates/Templat Programming http://en.wikipedia.org/wiki/Substitution_failure_is_not_an_error http://www.codeproject.com/Articles/3743/A-gentle-introduction-to-Template-Metaprogramming
- constexpr if
- SFINAE-based
- Expression templates: http://www.angelikalanger.com/Articles/Cuj/ExpressionTemplates/ExpressionTemplates.htm, https://www.cct.lsu.edu/~hkaiser/spring_2012/files/ExpressionTemplates-ToddVeldhuizen.pdf, див. також https://scholar.google.ca/citations?user=9vJnOMMAAAAJ&hl=en – сторінку автора, так статті простіше знайти буває.
- Мемоїзація в compile time: https://habr.com/ru/post/544960/.
- Brainfuck інтерпретатор на шаблонах: https://www.foonathan.net/2022/01/compile-time-codegen/.

- Спосіб деконструювати, поділити на частини – типи аргументів, тип результату тощо, вказівник на функцію за допомогою шаблонів (”Deconstructing function pointers in a C++ template”):
  - ◇ базовий варіант, підтримка noexcept, підтримка variadic-функцій.
  - ◇ ”Deconstructing function pointers in a C++ template, the calling convention conundrum”, ”trying to address the calling convention conundrum”, addressing the calling convention conundrum, returning to enable_if – на прикладі зоопарку угод про виклики Windows, всіляких cdecl та stdcall.
- ”Reordering C++ template type parameters for usability purposes, and type deduction from the future” – гарний приклад шаблонної магії.
- ”Writing a remove_all_pointers type trait, part 1” та ”part 2”.
- ”Open Content at CppCon: One API for Types and Values”
- ”How can I explicitly specialize a templated C++ constructor?”
  - ◇ ”How can I explicitly specialize a templated C++ constructor, follow-up notes”
- ”Using C++ type aliasing to avoid the ODR problem with conditional compilation, part 1”, ”part 2”.
- ”Compile-time C Compiler implemented as C++14 constant expressions” – компілятор C на шаблонах C++.
- ”The Global API Injection Pattern”

- `https://foonathan.net/2020/10/iife-metaprogramming/` – приклад використання лямбда-функцій для метапрограмування, отримання типу елементу в масиві чи контейнері:

```
template <typename T>
T type(); // no definition

template <typename Container>
using value_type_of = decltype([]{
        if constexpr (std::is_array_v<Container>)
            return type<std::remove_extent_t<Container>>();
        else
            return type<typename Container::value_type>();
    }());
```



- "Improving _Generic in C2y" – про інструменти, із задачами, схожими до шаблонів, в C.
- "Template Best Practices"